Pip Architecture — F1: one shared `buildAccountContext()` (the audit's "THE ONE THING")

FOLIOS · FOLIOSHQ.COM

Source: `docs/audit-2026-06-17.md` → X6 Pip Architecture Review. This is the audit's #1 recommendation. Goal: collapse the THREE drifting "account context" renderers into ONE shared builder so the "Pip knows X in chat but not in summaries" bug class becomes impossible by construction. ~1 week, ships with tests. Do the F4 cache wins alongside.

THE PROBLEM (verified file:line)

The same "what Pip knows about this account" is rendered by THREE independent functions that drift apart:

1. **Chat / Brief Me** — `src/lib/pipContext.js:223` `renderAccountFull(a, userId)`
   - called by `renderContextProse` (`pipContext.js:543`)
   - which `api/pip.js:381` calls (after `curateContext` at :380) for chat/brief modes.

2. **Meeting summarize** — `src/lib/pip.js` `summarizeDraftPip` assembles account context inline from many small blocks: `renderContactsBlock` (:248), `renderMeetingHistoryBlock` (:281), `renderAccountObjectiveBlock` (:324), `renderHealthTrendBlock` (:347), `renderSnapshotMetricsBlock` , `renderRecentUpdatesBlock` , `renderCommitmentsInBlock` , `renderPromiseLogBlock` , the operator block, etc. → `bp3Text` (:984) + `bp4Text` (:1003).
3. **Operator run** — `api/operator-run.js:71` `renderAccountContext(acc, meetings, tasks, contacts,`

`projects, updates, stateRow, snapshot, userId)` — called at :592.

Every parity fix in the 2026-06-17 session (globalPeople→chat, operator-state→summarize, waiting_on→briefs, health snapshots, ownership) was a MANUAL patch to one of these three. F1 is the structural cure.

THE GOAL

A single pure module — propose `src/lib/accountContext.js` (no Supabase import, so chat + summarize (client) AND operator-run (server) can all import it) — exporting:

- `buildAccountContext(account, bundle, opts)` → returns the canonical context, ideally as a structured object + a `renderAccountContextProse(ctx, opts)` so callers can choose prose (current behavior) and a future caller can persist the structured form (F2).
- `opts` carries surface flags (e.g. `{ surface: "chat"|"summarize"|"operator"|"brief", userId, includeOperatorRead, depth }`) so each surface gets the SAME data with only intentional per-surface trimming (the parity rule says wire a field to BOTH paths, not that every field goes to every surface — keep surface-appropriate omissions explicit).
- `bundle` = the per-account data already gathered: meetings, tasks (folio_tasks), contacts, projects (with .tasks), updates, snapshots/healthSnapshots, promiseStats, operator state row, globalPeople, glossary, profileProse, objective, systems, waiting_on, owner_user_id.

STEP-BY-STEP (route ONE surface at a time, snapshot-test between)

1. Build `accountContext.js` by lifting the UNION of fields the three renderers emit today (read all three carefully — they're listed above). Start from

`renderAccountFull` (it's the richest) as the base.

2. Add a **parity snapshot test** (`accountContext.test.js`): feed one fixed account bundle, assert the rendered prose contains every field each surface needs. This is the drift lock.
3. Route **chat/brief** first: `renderContextProse` calls `buildAccountContext` instead of `renderAccountFull` . Keep `curateContext` (the focus/list-mode selection) — F1 is about the per-account RENDER, not the curation. Verify chat tests still pass.
4. Route **summarize**: replace the bp3/bp4 inline block assembly in `summarizeDraftPip` with `buildAccountContext(..., {surface:"summarize"})` . Preserve the cache-breakpoint structure (BP1 static rules stay; the account context is the dynamic tail). Verify summarize tests.
5. Route **operator-run**: replace `renderAccountContext` with the shared builder (server import of the pure module — it must NOT import supabase).
6. Delete the now-dead per-surface renderers once all three route through the shared one.

ALREADY DONE — do NOT redo (the FEED lane is largely complete)

globalPeople→chat, operator-state→summarize, waiting_on→cadence brief + Brief Me, health snapshots + promiseStats into chat buildContext, ownership awareness, data-line guards, injection-resistance, digest parser v2. F1 should PRESERVE all of these (they become the shared builder's fields).

DO F4 ALONGSIDE (cheap money)

Prompt-cache discipline: keep the static system block byte-stable; the shared builder's output is the dynamic tail. Several endpoints already have `cache_control` (added 2026-06-17); audit the rest. Normalize/freeze `profileProse` per session (BP2 cache fragility, X6 F4).

THEN (later sessions, in the audit's order — NOT this pass)

F3 event-driven recompute (70-90% cost cut) → F2 persist the structured context into `folio_pip_account_state` → F5 real agent loop (tool_result round-trip, chat only) → F6 pgvector semantic recall (Pip summaries only; raw notes stay verbatim per Data Line Rule).

RISK CONTROLS

- Pure module, NO Supabase import (so operator-run server + client both use it).
- Route one surface at a time; run `npx vite build && npx vitest run && node scripts/check-guards.js` after each. Keep all 298 tests green.
- Don't change what reaches the model in a behavior-breaking way — this is a refactor to one source, not a context redesign. The parity snapshot test is the guard.
- Big refactor → work on the branch, do NOT deploy to main until Chris tests, then he ships.

WHERE THINGS LIVE

All current work is on `main` (deployed) and branch `claude/app-audit-strategy-hhcrz2` (same tip). Production is live + healthy. This F1 work should be its own session.